The Agora

The agora was the heart of the ancient Greek city, a public space where citizens gathered not just to trade, but to think, argue, and decide together. It was where the polis became more than a collection of individuals.

If no one person owns the understanding, how does a team build it together?

Every team has a place where the real thinking happens. Sometimes it's a meeting. More often it's a corridor conversation, a Slack thread, a one-on-one where someone finally said the thing they'd been sitting on for a week. The technical lead and the product manager talking through a problem before it becomes a ticket. The senior engineer pulling a junior aside to explain why the approach won't work. The offhand comment in a code review that reframes the whole feature. That thinking is the most valuable work the team does, and even in well-documented teams, part of it stays invisible, dependent on the right people being in the right conversation.

The Spec Session is what happens when a team makes that space intentional, the whole team converging on one spec before the agent runs. It's not a new idea: XP called it the planning game.[1] What's different is the stakes: where XP had an individual coder, the Spec Session has an agent. And the quality of the shared thinking before it runs determines everything that follows.

A reader running Scrum will have noticed that this is refinement. It is, and the activity is old. The name is different because the cadence is. One session, one spec, run when the work arrives rather than on a sprint boundary, and many of the teams this is for have no sprint to attach it to. What changed underneath is what waits at the end of it.

The individual coder was doing a second job nobody ever named. A human who doesn't understand what they're building gets uncomfortable, or stuck, and walks over to a desk and asks whether we meant this or that. That discomfort was a safety net no process designed: it

[1] Kent Beck, *Extreme Programming Explained: Embrace Change* — discussed, with citation, in the introduction; "the planning game" is one of its practices.

caught planning failures in week two instead of production. An agent has no such habit. It runs with its best interpretation or it stops, neither walking over. It can ask a question, but it asks the person who wrote the prompt, from inside that framing. The old process could afford an ambiguous spec because the implementer and the circuit breaker were the same person. The agent split them apart, and the Spec Session is what replaces the walk to the desk: the same catch, moved to the cheapest moment.

A better agent closes some of this. One that notices the ambiguity and asks the room instead of the author would restore the walk to the desk, and that would be worth having. It would not do the other thing. A question answered well leaves the understanding with whoever answered it, and the session exists because the team has to hold it, not because the agent has to receive it.

Before the Spec Session starts, there needs to be a reason to build anything at all. The business need and the user problem, as well as the organizational imperative, all equally important. The Spec Session aligns the team on what gets built and how, not whether it's worth building. Without a clear need, the best-run Spec Session produces a beautifully shared understanding of the wrong thing. This is a dedicated space and not a meeting: meetings have agendas, action items, owners. A Spec Session produces something different: a room full of people who hold the same picture of what's being built, why, and what done looks like. The document is the record of that convergence. It doesn't need everyone to read it the same way, only to keep pointing at the same thing.[2]

That's why the path is the goal. A team could be handed a perfectly written spec and skip the session. They'd own none of it, having inherited someone else's thinking rather than built their own. And when the agent runs and something unexpected surfaces, the team that built the understanding together knows what to do. The team that received the spec has to go find the person who wrote it.

I run climbing courses for children. At the start of every session we sit in a circle: everyone speaks, everyone listens, no rushing past. We reflect on the last time. We let the room breathe. We create what I think of as a shared Bewusstsein: a common consciousness of where everyone is before anyone starts climbing.[3]

[2]The sociological term is a boundary object — an artifact "plastic enough to adapt to local needs but robust enough to maintain a common identity" across groups (Susan Leigh Star and James Griesemer, "Institutional Ecology, 'Translations' and Boundary Objects," *Social Studies of Science*, 1989).

[3]Developmental psychology has a name for this: shared intentionality, the capacity for joint attention and joint goals that Michael Tomasello identifies as the distinctively human cognitive trick, present in infants before language (*Why We Cooperate*, MIT Press, 2009). Joint commitments carry norms: Gräfenhain, Behne, Carpenter,

Afterwards, the peer learning emerges. The child who felt heard speaks up on the wall. The one who listened offers help without being asked. The group becomes something that supports itself.

When we skip the circle, whether because there's no time, the energy is off, or someone rushes past it, the session feels disconnected. Individuals climbing in parallel rather than a group climbing together. The difference is subtle, real, and almost impossible to point to for anyone who hasn't felt it.

That's the Agora, the condition that makes a room more than a collection of individuals.

Software teams need the same thing, but as a disposition the team carries continuously and not as a ceremony: the habit of making thinking visible before acting on it, of listening before building, of checking where everyone is before assuming the team already knows. The climbing circle is one way to make that instinct visible. Some teams do it in how they write issues, or in the way a senior pauses before running the agent to ask whether the spec is clear. The form doesn't matter, but the instinct does. That's the moment of shared Bewusstsein before the agent runs.

The Spec Session is not a planning gate or waterfall with better tooling. The team still iterates. Still puts things in front of users. Still discovers that what got built isn't what was needed. The difference is that the team iterates with everyone holding the same picture, so when the direction turns out to be wrong, everyone understands why, and the correction is faster. In complex domains, where cause and effect only become visible in retrospect, the Spec Session doesn't replace iteration; it makes iteration less expensive by ensuring the team is wrong together.

Iterating is often the better instrument, and it is worth being exact about when. A probe beats a room where the loop is short, the cost of being wrong is bounded, and the wrongness is legible when it arrives. Under those conditions building the thing is the cheapest way to learn what it should have been, and running a session first is ceremony.

The room earns its place where the third condition fails. JoustMania's player history got built in full, storage and pipelines and recording and replay, and building it taught us nothing, because the feature was never wrong in its output. It had no referent. A controller ID is not a person, and no number of runs surfaces that. One question does.

Iteration surfaces wrong answers. It does not surface wrong questions. A build that answers the wrong question doesn't fail a user test, it passes, and solves nothing. Which is also when being wrong together is worth something: reality corrects a wrong answer for everyone at once, and it never corrects a wrong question at all.

---

and Tomasello showed that even three-year-olds take leave before abandoning a joint activity rather than walking away silently ("Young Children's Understanding of Joint Commitments," *Developmental Psychology* 45, 2009).

What that condition makes possible is more than shared understanding. It makes trust possible.

In climbing, trust has a physical form. Belaying, holding the rope that keeps another person safe, is the most literal expression of it. One person's life, in a meaningful sense, in another's hands. In my children's courses, we build to that point deliberately. The circle first. The shared awareness. The environment where everyone feels safe enough to be uncertain out loud. And then something that still surprises me happens. Five year olds belay each other. Supervised, of course, but the trust is real, the technique is real, and the care they take with each other is genuine. I have watched five year olds belay more attentively than adults I've seen on real rock, because the environment built the trust before the task required it. That's the standard software teams are measured against.

How many engineering teams have the trust required to say "I don't understand this spec"? To tell a senior their approach is wrong? To admit mid-implementation that something doesn't make sense and risk looking slow?

That trust is built deliberately, the same way the circle builds it before anyone climbs.

The hardest part is building this culture, regardless of the format.

A Spec Session breaks when people don't say what they actually think: the junior who spots a problem but doesn't feel safe raising it, the engineer who thinks the approach is over-engineered but defers to the senior, the concern that never gets voiced because the culture doesn't make it safe.

Psychological safety[4] is the prerequisite for a Spec Session. Without it, the team has a meeting where people perform agreement and the real thinking still happens in corridors afterward.

The Spec Session makes that culture structural rather than personal. It says: this is the time, this is the room, this is the moment where it's not just safe but expected to say what you think. The format protects the behavior that good leads were trying to create informally.

Spec Sessions can become the new endless grooming when the team circles the problem, surfaces every concern, explores every edge case, and never converges. Three hours later there's a rich document and no decision.

The antidote is a rotating session lead: someone whose job is to get to convergence, and to close the discussion when the understanding is good enough. Not perfect, just sufficient. Their job is also to protect the session from becoming the thing it was designed to replace.

---

[4]Amy Edmondson, "Psychological Safety and Learning Behavior in Work Teams," *Administrative Science Quarterly* (1999); later popularized via Google's Project Aristotle research.

The Spec Session produces minimum viable context: the agent runs without guessing, the team reviews without re-learning, the stakeholder recognizes what they asked for when they see it.

The session needs to surface three things before it closes.

What are we actually building. The specific behavior, the boundary conditions, the thing that would make a skeptic say "yes, that's done." The team needs to be able to describe it without looking at a document.

Why this approach and not another. Not for the audit trail, but for the moment six months from now when the system has changed and someone needs to know whether to revisit this or discard it. The why is what survives the what becoming obsolete.

What we're not building. The explicit scope boundary. The thing that's out of scope this iteration, that will be a different ticket. Without this, the agent fills the silence with reasonable guesses, wrong half the time.

When those three things are in the room and have been shared, challenged, and confirmed, the session is done.

The spec that comes out is not a user story wearing a new name, and it isn't acceptance criteria either. A story was a placeholder for a conversation that would happen during implementation, because the implementer could ask; the spec is the record of a conversation that already happened, because the implementer can't. Acceptance criteria are a contract, not an understanding.

The difference fits on half a page. What I actually typed to that OpenFeature contributor: this pull request is too large to review properly, split it into two for a better overview and an easier follow. What a session would have recorded instead: two pull requests, split by feature, not by module, each reviewable on its own. The shared logic stays in one place; neither duplicates it. Why: a better overview, and a follow that doesn't require holding two diffs in your head at once. Not doing: no restructuring of the existing module layout, no new abstractions for the shared code. Done when each pull request reviews cleanly by itself and the shared logic exists exactly once.

That's one task's record, not a form. Yours will look different, because your gaps are different.

Consensus is too high a bar and too slow to reach. The Spec Session produces consent, the decision-making distinction sociocracy has used for decades[5]. Nobody objects strongly enough to block, and that's enough. The team can move with a shared picture even when individuals would have made different choices. The document captures them. The team owns them. The agent can run.

[5] "Consent, not consensus" is a core distinction in sociocracy, formalized in Sociocracy 3.0 (sociocracy30.org).

The best room I have ever worked in had no walls at all.

Async Spec Planning is the same thing with a different form factor: for distributed teams, the session becomes a review request on the spec, and the merge condition is that the team has converged. It's slower than a room and faster than a document that gets written, inherited, and silently misunderstood.

But the async form carries a risk the room doesn't. What stops one developer from using an agent to write the spec proposal, and another from using an agent to review it? If both sides of the conversation are AI-mediated, the human understanding loop is bypassed before the code loop even starts. The spec merges. The understanding was never built. A merged spec that collected silent agreement is a session that never happened; if the understanding lives only in the artifact, the session failed.[6]

The full form, its mechanics and its mitigations, is Appendix D.

Tools are beginning to encode this instinct.[7] Birgitta Böckeler,[8] a Distinguished Engineer at ThoughtWorks, has analyzed the leading ones and identified the central gap: all of them assume a single developer does the requirements analysis. None address what happens when multiple people need to arrive at the same picture together. AWS's AI-DLC is the one exception, and it puts a room back: a ritual called mob elaboration, cross functional and hours long, before any agent runs.[9] Then it bets everything downstream on the artifacts that room produced, which is the bet this book argues against. A tool can encode the instinct, but it can't replace the room. A CONTEXT.md nobody owns is just a Confluence page. The record is not the theory, and the

[6]Chad Fowler's Deletion Test poses the same question at the artifact layer — imagine deleting the implementation, then ask whether the knowledge survives or the code was the only place it lived. Same structure, applied to code instead of a spec document. Chad Fowler, "The Deletion Test," *Phoenix Architectures* (aicoding.leaflet.pub).

[7]The landscape moves faster than a book can, so this is a snapshot rather than a survey. *Tools:* GitHub's Spec-Kit (github.com/github/spec-kit), the BMAD-Method (github.com/bmad-code-org/BMAD-METHOD), and Matt Pocock's AI Hero (aihero.dev), whose `/grill-with-docs` skill makes the Spec Session executable. *Concept:* "Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants" (arxiv.org/abs/2602.00180, 2026). *Risks:* ThoughtWorks' Technology Radar, Volume 33 (2025) flags spec-driven workflows becoming "lengthy" and "elaborate and opinionated" when the spec becomes the goal. *Closest external framing:* Kief Morris, "Humans and Agents in Software Engineering Loops," "Exploring Gen AI" (martinfowler.com), on humans "on the loop," building and maintaining the harness — Morris solves where humans sit in the process; this book solves what they need to do before it starts. A running version of this list lives at leftoftheloop.dev.

[8]Birgitta Böckeler, "Understanding Spec-Driven Development: Kiro, Spec-Kit, and Tessl," "Exploring Gen AI" (martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html). She analyzed Kiro, spec-kit, and Tessl.

[9]Raja SP, "AI-Driven Development Life Cycle: Reimagining Software Engineering," AWS DevOps & Developer Productivity Blog (aws.amazon.com/blogs/devops/ai-driven-development-life-cycle, 2025), and the AI-DLC Method Definition paper linked from it. The method assumes synchronous colocation — "a single room with a shared screen" — and persists every artifact it produces, from intent to test plan, as traceable context the AI references across the lifecycle.

understanding does not live in the markdown.

So start small. Smaller, even, than the team this book will argue for.

Grab a coworker before your next task. Define the plan together. Define the prompt together. Then hand it to the agent and let it run against what you agreed. If you want to change something mid-run, stop: that's another spec, not an edit to this one.

Let the output tell you whether the spec was good enough.

If it worked, iterate from there. You don't need to change your whole process from one day to the next.

If it didn't, ask why together, not each of you privately at your own desk. Where did the agent take the wrong turn? Would a better explanation have prevented it? Did it fill a gap you didn't know you'd left? Did you hand it so much context that the signal drowned? There are many ways a run can fail, and you and your coworker are the only ones who can tell which one you hit. A failed run is the cheapest teaching material you'll ever get, but only if the lesson lands in both heads. A lesson learned alone is the problem this book is about, in miniature.

Figure out what works, and build on it. One experiment at a time, one task at a time. That's the thing engineers are good at anyway: fixing broken things.

A fuller starting shape waits in the working template at the back.

Most teams struggle to make that space intentional. The thinking happens, but it happens in fragments. The Spec Session is the one place where the thinking that was happening anyway gets done together, gets documented, gets owned by the whole team. Not a ceremony added to a full calendar; the replacement for the fragments.

The Spec Session is where the work happens. Which leaves one question worth sitting with: why does it have to be people in the room at all?